

PHỤ LỤC SỔ TAY VẤN ĐÁP

# Từ điển Thuật ngữ

Giải thích mọi thuật ngữ trong QA\_DuyMinh

Dự án **StoryLens** · dành cho Đào Sỹ Duy Minh (23122041)

Mỗi thuật ngữ: định nghĩa ngắn gọn + cách nó xuất hiện trong dự án

**Môn học** CSC10011 – Công nghệ phần mềm cho hệ thống Trí tuệ Nhân tạo

**Nhóm** Nhóm 1 – StoryLens · HCMUS

## Contents

|          |                                                          |          |
|----------|----------------------------------------------------------|----------|
| <b>1</b> | <b>Quy trình &amp; Quản lý dự án (LN02–LN03)</b>         | <b>2</b> |
| <b>2</b> | <b>Kỹ nghệ yêu cầu &amp; Tài liệu (LN04–LN05)</b>        | <b>3</b> |
| <b>3</b> | <b>Kiến trúc &amp; Hệ thống (LN06)</b>                   | <b>3</b> |
| <b>4</b> | <b>AI/ML — Mô hình, Huấn luyện &amp; Đánh giá (LN07)</b> | <b>4</b> |
| <b>5</b> | <b>AI/ML — RAG, LLM &amp; Embedding</b>                  | <b>6</b> |
| <b>6</b> | <b>Kiểm thử &amp; Chất lượng (LN10–LN11)</b>             | <b>6</b> |
| <b>7</b> | <b>Bảo mật (Security)</b>                                | <b>7</b> |
| <b>8</b> | <b>DevOps &amp; Giám sát (Observability — LN11)</b>      | <b>8</b> |
| <b>9</b> | <b>Đạo đức AI (LN12)</b>                                 | <b>8</b> |

## 1. Quy trình & Quản lý dự án (LN02--LN03)

**Agile.** Triết lý phát triển phần mềm ưu tiên **lập nhanh, thích ứng thay đổi**, phần mềm chạy được hơn tài liệu đồ sộ. Ngược với Waterfall (làm tuần tự, chốt hết từ đầu). *Trong StoryLens*: chọn Agile vì dự án AI bất định cao, cần thử pipeline sớm.

**Waterfall (Thác nước).** Mô hình làm **tuần tự từng pha** (yêu cầu → thiết kế → code → test), pha sau chỉ bắt đầu khi pha trước xong. Rủi ro: phát hiện lỗi kiến trúc muộn thì sửa cực đắt.

**RUP (Rational Unified Process).** Quy trình lập chia làm **4 pha**: **Inception** (khởi tạo: vision, kế hoạch) → **Elaboration** (chi tiết hoá: use case, kiến trúc, prototype) → **Construction** (xây dựng: code, test) → **Transition** (chuyển giao: deploy, demo, nộp). *Trong StoryLens*: nhóm dùng “RUP rút gọn kết hợp Scrum”.

**Scrum.** Khung Agile chia việc thành các **Sprint** (chu kỳ ngắn cố định), mỗi cuối sprint có sản phẩm chạy được. *Trong StoryLens*: sprint = 2 tuần, 6 sprint tổng cộng.

**Sprint.** Một chu kỳ làm việc cố định (ở đây **2 tuần**). Cuối sprint phải có *increment* chạy được.

**Increment / Shippable increment.** **Phản sản phẩm tăng thêm** sau mỗi sprint, đủ hoàn chỉnh để chạy/demo được (“shippable” = giao được).

**Definition of Done (DoD).** Tiêu chí “**coi là xong**” thống nhất trước (vd: code + test + review + CI xanh). Tránh tranh cãi “xong chưa”.

**Product Backlog.** **Danh sách tất cả tính năng/việc cần làm** của cả dự án, sắp theo ưu tiên.

**Sprint Backlog.** Tập con của Product Backlog **chọn làm trong 1 sprint**.

**Story Point (SP).** **Đơn vị ước lượng độ khó tương đối** của một tính năng (theo độ phức tạp × rủi ro × nỗ lực), không phải giờ. *Trong StoryLens*: mốc Auth = 2 SP, Pipeline AI = 8 SP; tổng backlog **135 SP**.

**Planning Poker.** Kỹ thuật ước lượng: cả nhóm cùng “**bóc bài**” gán **Story Point**, thảo luận đến khi thống nhất — giảm thiên lệch cá nhân.

**Velocity (Vận tốc).** **Số Story Point nhóm hoàn thành trung bình mỗi sprint**. *Trong StoryLens*:  $\approx 22\text{--}23$  SP/sprint (đo thật: 23/24/23/26/22/17).

**COCOMO / KLOC.** COCOMO = mô hình ước lượng chi phí dựa trên **KLOC** (nghìn dòng code). Nhóm **không dùng** vì AI/RAG không đoán được KLOC và dòng code không phản ánh độ khó.

**Kanban.** Bảng trực quan hoá công việc theo cột trạng thái (To Do → In Progress → Reviewing → Done). Giúp thấy ngay việc đang tắc ở đâu.

**Burndown chart.** Biểu đồ **lượng công việc còn lại (Story Point) giảm dần theo thời gian**; so đường thực tế với đường lý tưởng để phát hiện chậm tiến độ.

**WBS (Work Breakdown Structure).** **Sơ đồ phân rã công việc lớn thành các task nhỏ** quản lý được. *Trong StoryLens*: 31 task T1.1–T6.5.

**Gantt chart.** Biểu đồ **thanh ngang thời gian**: mỗi task một thanh trải trên các tuần, kèm mốc (milestone).

**Milestone / Build / Release.** **Mốc quan trọng** đánh dấu một phiên bản giao được. *Trong StoryLens*: 3 mốc Build (Build 1 backend+OCR, Build 2 full pipeline, Build 3 final).

**Staff Allocation (phân bổ nhân sự).** Bảng **ai làm task gì ở tuần nào** để phát hiện người quá tải / nhàn rỗi.

**Risk Register (sổ rủi ro) & RE.** Danh sách rủi ro kèm **RE = Probability × Loss** (khả năng xảy ra × mức thiệt hại), dùng để **xếp hạng ưu tiên xử lý**. *Trong StoryLens*: top: R2 quota, R5 bản quyền (RE = 5625).

**Weekly Scrum / Daily Standup.** Họp ngắn định kỳ trả lời **3 câu hỏi**: tuần/hôm qua làm gì? đang vướng gì? sắp làm gì?

**Flat team (đội hình phẳng).** Cơ cấu **không phân cấp cứng**, quyết định kỹ thuật thảo luận tập thể. Không mâu thuẫn với vai trò PM (PM lo điều phối, không áp đặt giải pháp).

**Pull Request (PR) & Code Review.** PR = **đề nghị gộp code** từ nhánh phụ vào nhánh chính; Code Review = **ít nhất 1 người khác đọc duyệt** trước khi gộp. Là công chất lượng của nhóm.

## 2. Kỹ nghệ yêu cầu & Tài liệu (LN04--LN05)

**Functional Requirement (FR — yêu cầu chức năng).** Mô tả **hệ thống phải làm gì** (hành vi). *Trong StoryLens:* F01–F09, vd F03 pipeline AI, F05 hỏi–đáp RAG.

**Non-Functional Requirement (NFR — yêu cầu phi chức năng).** Ràng buộc **chất lượng** (hiệu năng, bảo mật, độ sẵn sàng). *Trong StoryLens:* phản hồi < 3s, xử lý AI < 30s/trang.

**Use Case.** Mô tả **một luồng người dùng đạt được mục tiêu** khi tương tác hệ thống. *Trong StoryLens:* UC01 upload–dịch, UC02 đọc overlay.

**Actor (tác nhân).** Người/hệ thống **bên ngoài** tương tác với hệ thống. *Trong StoryLens:* Người đọc, Quản trị viên, và tác nhân phụ “AI Module + Gemini”.

<<include>> / <<extend>>. Quan hệ giữa use case: **include** = luôn gọi kèm (bắt buộc); **extend** = chỉ xảy ra có điều kiện (tuỳ chọn). *Trong StoryLens:* UC “Dịch batch” include “Trừ credit”.

**Vision Document.** Tài liệu **tầm nhìn**: vấn đề, định vị sản phẩm, người dùng — ở mức nghiệp vụ.

**SRS (Software Requirements Specification).** **Đặc tả yêu cầu phần mềm** chi tiết (FR + NFR), ở mức hệ thống.

**SAD (Software Architecture Document).** **Tài liệu kiến trúc**: các thành phần, quan hệ, quyết định thiết kế.

**SDP (Software Development Plan).** **Kế hoạch phát triển**: lịch, nhân sự, rủi ro, ngân sách.

**Persona.** **Chân dung người dùng mẫu** (tuổi, nhu cầu, kỳ vọng) để thiết kế bám nhu cầu thật. *Trong StoryLens:* sinh viên, học sinh THPT, content creator.

**RTM (Requirements Traceability Matrix).** Ma trận **truy vết**: mỗi Yêu cầu ↔ Use Case ↔ Module ↔ Test case — đảm bảo mọi yêu cầu đều có code và test.

## 3. Kiến trúc & Hệ thống (LN06)

**Microservice.** Kiến trúc chia hệ thống thành **nhiều dịch vụ nhỏ triển khai độc lập**. *Trong StoryLens:* 3 dịch vụ: Frontend, Backend, AI Module.

**Monolith.** Ngược với microservice: **gộp tất cả vào một khối**. Đơn giản nhưng khó scale từng phần.

**Pipe-and-Filter (đường ống & bộ lọc).** Kiến trúc **dữ liệu chảy qua chuỗi bộ lọc** độc lập, mỗi bộ lọc biến đổi rồi truyền tiếp. *Trong StoryLens:* pipeline YOLO → OCR → LaMa → Gemini → render.

**Client–Server.** Máy khách (trình duyệt) gửi yêu cầu, máy chủ xử lý và trả về.

**Layered (phân tầng).** Chia code theo tầng (router → service → database). *Trong StoryLens:* backend: business logic ở services/, không ở router.

**4+1 View & Process View.** Cách mô tả kiến trúc bằng 5 góc nhìn (Logical, Development, Process, Physical/Deployment, + Use-Case). **Process View** = góc nhìn **tiến trình/luồng chạy đồng thời** (thread, hàng đợi).

**ADR (Architecture Decision Record).** **Bản ghi quyết định kiến trúc**: vấn đề / quyết định / lý do / phương án khác / hệ quả. Giúp giải thích “vì sao chọn thế”.

**REST API.** Kiểu giao tiếp qua HTTP dùng các phương thức GET/POST... trả về JSON. *Trong StoryLens:* backend FastAPI expose REST.

**WebSocket.** Kênh hai chiều, giữ kết nối liên tục giữa client và server (khác REST chỉ hỏi–đáp một lần). *Trong StoryLens:* truyền tiến trình dịch thời gian thực.

**Polling fallback.** Khi WebSocket lỗi, client hỏi lại server định kỳ (polling) để không mất cập nhật.

**Semaphore / BoundedSemaphore.** Cơ chế giới hạn số luồng chạy đồng thời. *Trong StoryLens:* giới hạn số ảnh AI xử lý cùng lúc để không tràn RAM.

**Cold start.** Độ trễ lần gọi đầu khi dịch vụ vừa “thức dậy” sau khi ngủ (free tier). *Trong StoryLens:* Render/HF Spaces ngủ khi rảnh.

**Keep-alive.** Ping định kỳ để giữ dịch vụ không ngủ, giảm cold start.

**CDN (Content Delivery Network).** Mạng máy chủ phân phối nội dung gần người dùng cho nhanh. *Trong StoryLens:* frontend qua Vercel CDN.

**Docker / Container / Image.** Đóng gói ứng dụng + phụ thuộc vào một “hộp” chạy giống nhau ở mọi nơi. Image = khuôn, Container = bản chạy. *Trong StoryLens:* AI Module image ~4GB.

**Supabase / PostgreSQL.** Supabase = nền tảng backend dựng trên PostgreSQL (hệ quản trị CSDL quan hệ) kèm Auth + Storage. *Trong StoryLens:* DB chính của StoryLens.

## 4. AI/ML --- Mô hình, Huấn luyện & Đánh giá (LN07)

**OCR (Optical Character Recognition).** Nhận dạng ký tự từ ảnh thành văn bản. *Trong StoryLens:* manga-ocr đọc chữ Nhật trong bong bóng.

**Object Detection / Bounding Box.** Phát hiện vật thể và khoanh nó bằng khung chữ nhật (bounding box). *Trong StoryLens:* YOLOv8 khoanh bong bóng thoại.

**YOLOv8.** Họ mô hình phát hiện đối tượng thời gian thực (You Only Look Once). YOLOv8n = bản “nano” nhẹ (~3M tham số), chạy được CPU.

**Pretrained model / Base model.** Mô hình đã huấn luyện sẵn trên dữ liệu lớn, dùng làm điểm xuất phát. *Trong StoryLens:* kha-white/manga-ocr-base.

**Fine-tune (tinh chỉnh).** Huấn luyện tiếp một base model trên dữ liệu chuyên biệt của mình để giỏi hơn ở tác vụ cụ thể. *Trong StoryLens:* fine-tune manga-ocr trên Manga109-s → CER 6,1%.

**VisionEncoderDecoderModel.** Kiến trúc encoder đọc ảnh + decoder sinh văn bản — nền của manga-ocr.

**Dataset split (train/val/test).** Chia dữ liệu: train (học), val (chỉnh siêu tham số), test (đánh giá cuối, chưa từng thấy). *Trong StoryLens:* 80/10/10 = 97.602 / 11.476 / 14.130.

**Split-by-title / Data leakage.** Chia dữ liệu theo đầu truyện để cùng một truyện không nằm cả ở train lẫn test — chống rò rỉ dữ liệu (test bị “học lỏm” → điểm ảo).

**Seed.** Số khởi tạo ngẫu nhiên cố định để kết quả tái lập được. *Trong StoryLens:* seed = 42.

**Epoch.** Một lượt mô hình học hết toàn bộ tập train. *Trong StoryLens:* 8 epochs.

**Learning rate (lr).** Bước cập nhật trọng số mỗi lần học — lớn quá thì loạn, nhỏ quá thì chậm. *Trong StoryLens:* lr = 3e-5.

**AdamW.** Thuật toán tối ưu phổ biến để cập nhật trọng số (Adam + weight decay đúng cách).

**Cosine schedule / Warmup.** Cách giảm dần learning rate theo đường cong cosin; warmup = tăng dần lr ở đầu để ổn định. *Trong StoryLens:* warmup 5%.

**Label smoothing.** Kỹ thuật làm “mềm” nhãn (không tuyệt đối 100%) để mô hình bớt tự tin thái quá, giảm overfit. *Trong StoryLens:* 0.1.

**Weight decay.** Phạt trọng số lớn để chống overfit (regularization). *Trong StoryLens:* 0.01.

**fp16 (mixed precision).** Dùng số thực 16-bit thay 32-bit khi train → nhanh hơn, tốn ít bộ nhớ GPU.

**Early stopping / Patience.** Dừng train sớm khi chỉ số val không cải thiện sau “patience” lượt → chống overfit. *Trong StoryLens:* patience 3, theo CER val nhỏ nhất.

**Checkpoint.** Bản lưu trọng số tại một thời điểm; chọn checkpoint tốt nhất theo val. *Trong StoryLens:* best.pt.

**Ablation study.** So sánh có/không một yếu tố để chứng minh yếu tố đó có tác dụng. *Trong StoryLens:* PA2 vs PA3 (12,7% → 6,1%) là ablation về cấu hình huấn luyện.

**Metadata / Artifact.** Sản phẩm phụ do quá trình train sinh ra (file chỉ số, biểu đồ). *Trong StoryLens:* metadata.json chứa test\_cer thật để xác minh.

**IoU (Intersection over Union).** Diện tích giao / diện tích hợp giữa khung dự đoán và khung thật — đo độ khớp. Ngưỡng  $\geq 0.5$  tính là đúng.

**Precision (Độ chính xác).**  $\frac{TP}{TP + FP}$  — trong số khung mô hình vẽ ra, bao nhiêu % đúng. *Trong StoryLens:* 97,3%.

**Recall (Độ bao phủ).**  $\frac{TP}{TP + FN}$  — trong số khung thật sự có, mô hình tìm được bao nhiêu %. *Trong StoryLens:* 93,8%.

**TP / FP / FN.** True Positive (đúng–dương), False Positive (báo nhầm), False Negative (bỏ sót).

**PR Curve / AP / mAP.** PR Curve = đường Precision–Recall; AP = diện tích dưới đường đó (1 lớp); mAP = trung bình AP các lớp. *Trong StoryLens:* mAP@0.5 = 95,3%.

**mAP@0.5 vs mAP@0.5:0.95.** @0.5 = tại ngưỡng IoU 0.5; @0.5:0.95 = trung bình nhiều ngưỡng 0.5 → 0.95 (nghiêm ngặt hơn). *Trong StoryLens:* 95,3% vs 84,4%.

**CER (Character Error Rate).**  $\frac{S + D + I}{N}$  — tỉ lệ lỗi ký tự của OCR ( $S$  thay thế,  $D$  xóa,  $I$  chèn,  $N$  tổng ký tự chuẩn). *Trong StoryLens:* 6,1%.

**Char-Accuracy.**  $1 - CER$  = độ chính xác ký tự. *Trong StoryLens:* 93,9%.

**Exact-Match.** Tỉ lệ dự đoán khớp 100% cả chuỗi — chỉ số nghiêm ngặt hơn CER. *Trong StoryLens:* 67,5%.

**Levenshtein distance.** Số thao tác tối thiểu (thêm/xóa/thay) biến chuỗi này thành chuỗi kia — nền của CER.

**In-distribution / Out-of-distribution (OOD).** Dữ liệu đúng / khác phân phối huấn luyện. *Trong StoryLens:* CER 6,1% là in-distribution (Manga109-s); manga phong cách khác là OOD, chưa đo số.

**Distribution bias.** Thiên lệch do dữ liệu huấn luyện nghiêng về một loại. *Trong StoryLens:* Manga109-s chủ yếu manga Nhật → kém với manhwa Hàn/manhua Trung.

**Concept drift.** Dữ liệu thực tế thay đổi theo thời gian khiến mô hình kém dần sau khi deploy.

**MLOps / Model Monitoring.** Vận hành ML sau deploy: theo dõi chất lượng mô hình theo thời gian (drift), retrain khi cần. *Trong StoryLens:* chưa có (đang MVP); roadmap W&B/MLflow.

**W&B (Weights & Biases) / MLflow.** Công cụ theo dõi thí nghiệm ML & registry mô hình. Hướng phát triển của nhóm.

**LaMa / Inpainting.** **Inpainting** = “vá” vùng ảnh bị xoá cho liền nền; **LaMa** là mô hình inpainting. *Trong StoryLens:* xoá chữ gốc khỏi bong bóng trước khi vẽ chữ Việt.

## 5. AI/ML --- RAG, LLM & Embedding

**LLM (Large Language Model).** Mô hình ngôn ngữ lớn sinh/hiểu văn bản. *Trong StoryLens:* **Gemini 2.5 Flash** để dịch & trả lời hỏi–đáp.

**Prompt Engineering.** Thiết kế câu lệnh (prompt) đưa vào LLM để có kết quả tốt. *Trong StoryLens:* Minh viết prompt dịch cả trang giữ ngữ cảnh.

**Context Injection.** Nhồi thêm ngữ cảnh (cả trang + glossary xung hô) vào prompt để LLM dịch nhất quán, không rời rạc.

**Hallucination (ảo giác).** LLM “bịa” thông tin nghe hợp lý nhưng sai. *Trong StoryLens:* giảm bằng RAG (bám nguồn) + human-in-the-loop.

**Human-in-the-loop.** Con người trong vòng lặp kiểm/sửa kết quả AI. *Trong StoryLens:* Studio QC cho dịch giả duyệt/sửa từng bong bóng.

**Embedding / Vector.** Biểu diễn văn bản thành dãy số (vector) sao cho nghĩa gần nhau thì vector gần nhau. *Trong StoryLens:* gemini-embedding-001, 768 chiều.

**Cosine Similarity.** Đo độ giống nhau giữa hai vector bằng góc giữa chúng (1 = giống, 0 = không liên quan). *Trong StoryLens:* tìm đoạn thoại liên quan câu hỏi.

**Top-k Retrieval.** Lấy k đoạn giống nhất làm ngữ cảnh cho LLM.

**RAG (Retrieval-Augmented Generation).** Truy hồi đoạn liên quan rồi sinh câu trả lời bám vào đó → AI trả lời đúng nội dung, có nguồn trích, ít bịa. *Trong StoryLens:* UC04 hỏi–đáp.

**Vector Database (pgvector / ChromaDB / FAISS).** CSDL lưu & tìm vector theo độ tương đồng. **pgvector** = extension vector cho PostgreSQL. *Trong StoryLens:* đã chọn pgvector thay ChromaDB (bớt 1 dịch vụ).

**Fallback (M2M100 / NLLB).** **Phương án dự phòng:** khi Gemini lỗi/hết quota thì chuyển sang mô hình dịch mã nguồn mở (M2M100, NLLB).

**Key Rotation / Key Pool / RPD / Quota / Throttling.** Xoay vòng nhiều API key để né giới hạn; **RPD** = Requests Per Day; **Quota** = hạn mức; **Throttling** = tự giảm tần suất gọi. *Trong StoryLens:* chống hết quota Gemini free tier.

**Graceful Degradation.** Suy giảm êm: một phần lỗi thì hệ thống trả kết quả một phần thay vì sập toàn bộ. *Trong StoryLens:* VLM/forum tắt thì trả rỗng, không crash.

## 6. Kiểm thử & Chất lượng (LN10--LN11)

**Verification vs Validation (V&V).** **Verification** = “làm sản phẩm đúng đặc tả” (unit test); **Validation** = “làm đúng sản phẩm khách cần” (UAT, E2E).

**Unit / Component / Integration / System / Acceptance test.** Các cấp kiểm thử: đơn vị hàm → thành phần UI → tích hợp nhiều module → toàn hệ → nghiệm thu người dùng.

**UAT (User Acceptance Testing).** Kiểm thử chấp nhận bởi người dùng thật. *Trong StoryLens:* mời 3–5 người thử.

**E2E (End-to-End) test.** Kiểm thử **cả hành trình người dùng** trên trình duyệt thật (đăng nhập → upload → đọc). *Trong StoryLens:* Playwright.

**Playwright / Vitest / pytest / respx.** Công cụ test: **Playwright** (E2E trình duyệt), **Vitest** (unit frontend), **pytest** (backend), **respx** (giả lập HTTP).

**Katalon Studio.** Công cụ test **kéo-thả, ghi-phát lại** (record&playback), xuất báo cáo trực quan. *Trong StoryLens:* dùng cho PA5 (UC01, UC02).

**Regression Testing (kiểm thử hồi quy).** **Chạy lại test cũ** để chắc code mới không làm hỏng tính năng cũ. *Trong StoryLens:* tự động trong CI.

**Test Pyramid.** Nguyên tắc: **nhiều unit test, ít E2E** (unit rẻ/nhanh, E2E đắt/chậm).

**Smoke test.** **Kiểm tra nhanh** hệ thống “còn thở” sau deploy (chạy được các chức năng chính).

**CI/CD (Continuous Integration / Delivery).** **Tự động build + test + deploy** mỗi khi push code. *Trong StoryLens:* GitHub Actions **5 job song song**.

**CI Quality Gate.** **Chốt chặn:** chỉ merge/deploy khi CI xanh 100%.

**Linter (ESLint).** Công cụ **phân tích tĩnh** bắt lỗi cú pháp/định dạng/code dư (biến không dùng). Đồng bộ style 6 người.

**Typecheck (tsc --noEmit).** TypeScript **kiểm tra kiểu dữ liệu** mà không xuất file JS (--noEmit). Bắt lỗi bất khớp kiểu giữa frontend-backend trước khi chạy.

**Static Analysis (phân tích tĩnh).** Phân tích code **mà không chạy chương trình** (Linter + Typecheck thuộc nhóm này).

**Equivalence Partitioning / Boundary Value Analysis.** Kỹ thuật thiết kế test: **chia miền giá trị thành nhóm tương đương và thử ở biên** (vd file 0/1/10/11 MB).

**Defect log.** **Nhật ký lỗi** tìm được (found → fixed → retested). *Trong StoryLens:* hiện 61/61 Pass, 0 defect (điểm cần thử).

## 7. Bảo mật (Security)

**XSS (Cross-Site Scripting).** Kẻ tấn công **chèn JavaScript độc** vào trang để đánh cắp dữ liệu. Chống bằng HttpOnly cookie + CSP.

**CSRF (Cross-Site Request Forgery).** **Giả mạo yêu cầu** từ trang khác nhân danh người dùng đã đăng nhập. Chống bằng SameSite cookie + token.

**SSRF (Server-Side Request Forgery).** Lừa **server tự gửi request tới địa chỉ nội bộ**. *Trong StoryLens:* chống bằng domain allowlist ở scraper.

**HttpOnly / Secure / SameSite Cookie.** Thuộc tính cookie: **HttpOnly** (JS không đọc được → chống XSS), **Secure** (chỉ gửi qua HTTPS), **SameSite** (chống CSRF). *Trong StoryLens:* token đăng nhập đặt trong HttpOnly cookie.

**RBAC (Role-Based Access Control).** **Phân quyền theo vai trò** (user / admin) ở tầng API.

**RLS (Row Level Security).** **Bảo mật mức dòng** ở database: mỗi truy vấn tự thêm điều kiện `owner_id = auth.uid()` → user A không đọc được dữ liệu user B.

**IDOR (Insecure Direct Object Reference).** Lỗi hỏng **truy cập trực tiếp tài nguyên của người khác** bằng cách đổi ID. *Trong StoryLens:* đóng bằng owner-scoped RAG + RLS.

**Rate Limiting / SlowAPI.** Giới hạn số request mỗi khoảng thời gian để chống lạm dụng. *Trong StoryLens:* SlowAPI theo endpoint.

**CAPTCHA / Turnstile.** Cơ chế phân biệt người với bot. *Trong StoryLens:* Cloudflare Turnstile ở đăng ký, quên mật khẩu.

**Security Headers.** Các HTTP header bảo vệ gắn vào mọi phản hồi: **CSP** (giới hạn nguồn script), **HSTS** (ép HTTPS), **X-Frame-Options** (chống clickjacking), **X-Content-Type-Options** (chống MIME sniffing).

**MIME Sniffing / Spoofing.** Trình duyệt tự đoán loại file (sniffing); kẻ xấu giả loại file (spoofing, vd script đổi tên .jpg). Chống bằng Pillow verify() + header nosniff.

**Domain Allowlist.** Danh sách miền được phép; mọi URL ngoài danh sách bị chặn. *Trong StoryLens:* chống SSRF khi import ảnh.

**Idempotency Key.** Khóa bất biến đảm bảo gửi lại request nhiều lần chỉ thực thi một lần (chống trừ tiền 2 lần). *Trong StoryLens:* thanh toán Stripe.

**Defense in Depth.** Phòng thủ nhiều lớp — nếu một lớp thùng vẫn còn lớp khác.

**GDPR Export.** Cho người dùng tải về toàn bộ dữ liệu cá nhân của mình (tuân thủ quyền riêng tư).

**Audit Log.** Nhật ký kiểm toán: ghi “ai làm gì, lúc nào, trên dữ liệu nào” — phục vụ điều tra tranh chấp. *Trong StoryLens:* admin actions vào bảng audit\_logs.

## 8. DevOps & Giám sát (Observability --- LN11)

**Sentry.** Công cụ giám sát lỗi (error monitoring): bắt exception runtime, gửi alert kèm stack trace. Cấu hình send\_default\_pii=False để không log thông tin cá nhân.

**OpenTelemetry (OTel).** Chuẩn distributed tracing: theo dõi một request đi qua nhiều dịch vụ để tìm cổ chai (bottleneck).

**Span / Trace.** Span = một đoạn thời gian của một bước; Trace = chuỗi span của cả request.

**Structured Logging.** Ghi log có cấu trúc (timestamp, level, module, request-id) thay vì print() tự do.

**Request ID (X-Request-ID / UUID).** Mã định danh duy nhất gắn cho mỗi request; lọc log theo mã này là ra toàn bộ hành trình. UUID = mã 128-bit gần như không trùng.

**Health Check (/health, /health/deep).** Endpoint báo sức khỏe hệ thống; deep kiểm tra cả Supabase/Gemini/AI Module.

**Readiness Probe / Zero-downtime.** Nền tảng ping bản mới trước khi chuyển traffic; nếu không khỏe thì giữ bản cũ → không gián đoạn.

**PITR (Point-in-Time Recovery).** Khôi phục CSDL về một thời điểm bất kỳ nhờ log liên tục. *Trong StoryLens:* Supabase PITR 7 ngày.

**Vercel / Render / HuggingFace Spaces.** Nền tảng deploy: Vercel (frontend), Render (backend Docker), HF Spaces (AI Module GPU/CPU).

## 9. Đạo đức AI (LN12)

**Opacity (Mờ đục / Hộp đen).** AI khó giải thích vì sao ra kết quả đó. *Trong StoryLens:* giải bằng transparency: progress bar mỗi bước + structured log + visualize bounding box.

**Transparency / Explainability / Accountability.** Minh bạch (cho thấy đang làm gì) / giải thích được / có người chịu trách nhiệm khi sai.

**Artificial Stupidity.** AI mắc lỗi “ngớ ngẩn” (OCR đọc sai, YOLO khoanh nhầm). *Trong StoryLens:* giảm bằng Gemini self-correction + Studio QC.

**AI Bias / Fairness.** Thiên lệch & công bằng của mô hình do dữ liệu. *Trong StoryLens:* Manga109-s nghiên cứu manga Nhật → ghi nhận giới hạn + bổ sung dataset.

**Privacy & Surveillance.** Riêng tư & giám sát. *Trong StoryLens:* thư viện mặc định Private, Sentry không log PII.

**DMCA Take-down.** Quy trình gỡ nội dung vi phạm bản quyền theo yêu cầu chủ sở hữu. *Trong StoryLens:* trong roadmap.

**Manga109-s.** Bộ dữ liệu học thuật 109 đầu manga Nhật (Aizawa Lab, ĐH Tokyo), có giấy phép nghiên cứu. *Trong StoryLens:* dùng train YOLO + OCR.

**“Principles alone cannot guarantee ethical AI” (LN12).** Chỉ tuyên bố nguyên tắc là chưa đủ — phải có thiết kế kỹ thuật enforce (RLS, Sentry config, allowlist) mà dev khác không bypass được.